AnyDoc: A Hybrid Retrieval-Augmented Generation System — Architecture, API Reference, and Production Deployment

Independent project · Source: github.com/Tarunvarma07/AnyDoc_chat

Abstract

AnyDoc is a document question-answering system combining dense and sparse retrieval, cross-encoder reranking, and LLM-based query rewriting behind a versioned REST API, with a Streamlit operator dashboard as a reference client. Beyond the retrieval architecture, this report documents the system as a deployable service: a complete API reference for its four endpoints, the Docker/CI/CD pipeline that ships it to a hosting platform, and three production incidents encountered while deploying it - an out-of-memory crash traced to a transitive PyTorch dependency, a request-timeout failure traced to an O(n) LLM call pattern, and a guardrail false positive traced to a fuzzy-matching library - each with root-cause diagnosis, a verified fix, and a regression test. We evaluate the retrieval stack on a 105-document adversarial benchmark and report hit-rate results for dense-only versus hybrid retrieval.

1. Introduction

Retrieval-Augmented Generation (RAG) systems ground large language model outputs in retrieved documents, reducing hallucination and enabling up-to-date, source-attributable answers (Lewis et al., 2020). A common failure mode of RAG implementations that rely solely on dense vector similarity is poor recall on queries that hinge on exact tokens - identifiers, error codes, version strings - which dense embeddings can under-weight relative to broader semantic similarity. AnyDoc addresses this by combining dense retrieval with a classical sparse retriever (BM25) and fusing the two result sets with Reciprocal Rank Fusion (Cormack et al., 2009), then applying a cross-encoder reranker (Nogueira and Cho, 2019) to the fused candidate pool before generation.

A retrieval pipeline is only half the system. This report treats AnyDoc as a service with a public contract and a deployment lifecycle, not just an algorithm: Section 2 covers the retrieval and generation architecture; Section 3 is a complete API reference; Section 4 covers the deployment and CI/CD pipeline; Section 5 documents three real production incidents and their fixes; Section 6 reports evaluation results; Section 7 covers testing strategy; Sections 8-10 cover related work, limitations, and conclusions.

The contributions are:

- A hybrid dense + sparse retriever with RRF fusion and an event-driven index that stays consistent as documents are added.
- A cross-encoder reranking stage running on an ONNX runtime (not PyTorch) for a materially smaller production memory footprint.
- An LLM query-rewriting stage that decouples the retrieval query from the generation query.
- SSRF- and MIME-hardened ingestion for URL and file sources.
- A versioned REST API (Section 3) and a Docker/CI/CD deployment pipeline (Section 4) - the system runs as a deployed service, not only as a local script.
- A reproducible, adversarial evaluation harness comparing dense-only and hybrid retrieval hit rates.

2. System Architecture

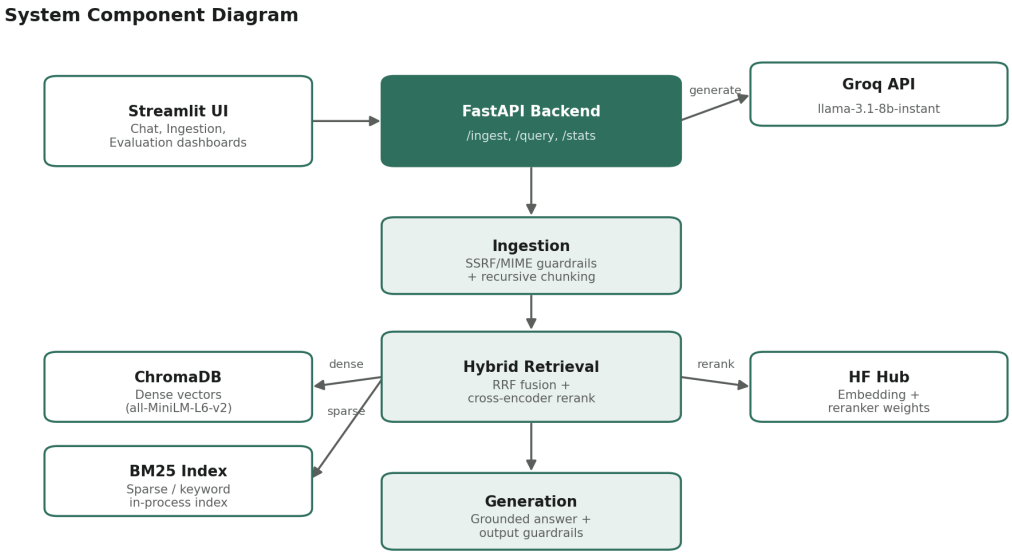


Figure 1. System component diagram: the Streamlit UI talks only to the FastAPI backend, which owns ingestion, hybrid retrieval, and generation.

2.1 Ingestion and Guardrails

Documents are ingested from either a file upload (PDF, DOCX, TXT, CSV) or a URL. File uploads are validated by size and by inspecting file magic bytes rather than trusting the declared MIME type or extension. URL ingestion resolves the target hostname and rejects loopback, private, and link-local addresses, including an explicit block on the AWS metadata IP (169.254.169.254). Because redirects can be used to bypass an initial URL check, each redirect hop is re-validated before being followed, and web crawling respects *robots.txt*. Ingested text is split with a recursive character splitter (500-character chunks, 50-character overlap), chosen over a naive fixed-width splitter because it preferentially breaks on paragraph and sentence boundaries, reducing fragment and mid-sentence-cut rates.

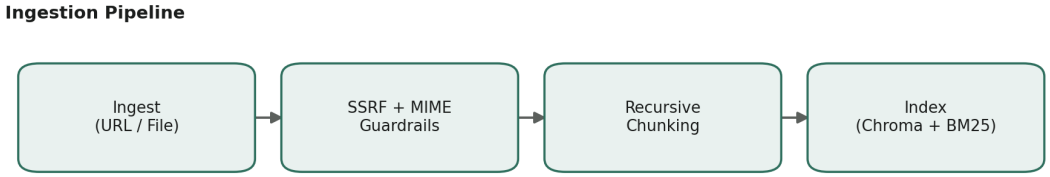


Figure 2. Ingestion pipeline: every document passes through SSRF/MIME guardrails before chunking and indexing.

2.2 Indexing

Each chunk is embedded with a local Sentence-Transformers model (`sentence-transformers/all-MiniLM-L6-v2`, Reimers and Gurevych, 2019) run on fastembed's ONNX runtime, and stored in a persistent ChromaDB collection. A parallel BM25 index (Robertson and Zaragoza, 2009) is built over the same chunk set. An observer callback on the embedding store triggers a BM25 rebuild whenever new documents are added, preventing the two indexes from silently drifting out of sync.

2.3 Retrieval Pipeline

At query time, an LLM-based query rewriter first expands the user's natural-language question into a keyword-enriched search query. Retrieval then proceeds against the rewritten query only; the original question is preserved separately for the generation stage. The hybrid retriever pulls the top candidates from both the dense

(Chroma) and sparse (BM25) indexes and fuses them with Reciprocal Rank Fusion:

$$RRF(d) = \Sigma_{r \in R} \; 1 / (k + rank_r(d))$$

where R is the set of ranked lists (dense, sparse), $rank_r(d)$ is document d's rank in list r, and k = 60. When a reranker is attached, the retriever widens its pre-fusion candidate pool to 3× the final top-N before fusing. The cross-encoder (an ONNX export of `cross-encoder/ms-marco-MiniLM-L-6-v2`) then scores each (query, document) pair jointly, and the top-N by cross-encoder score are passed to generation.

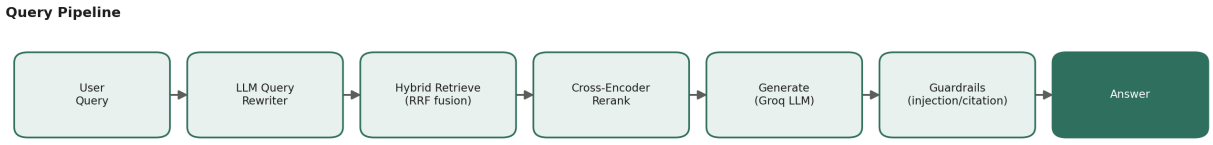


Figure 3. Query pipeline: retrieval runs against the rewritten query; generation and guardrails run against the user's original question.

2.4 Generation and Output Guardrails

The retrieved chunks are formatted into a numbered context block and passed to an LLM (`llama-3.1-8b-instant` via the Groq API) with a strict grounding prompt. Before generation, the input query is screened for prompt-injection patterns and profanity; after generation, the output is screened for profanity and for hallucinated citations (bracketed indices that do not correspond to any retrieved document). Any guardrail trigger returns a distinct refusal message rather than the model's raw output.

3. API Reference

AnyDoc exposes a small REST API (FastAPI, JSON over HTTP) that the Streamlit dashboard consumes as an ordinary client - any other client can integrate against the same contract. All endpoints are unauthenticated in the current deployment; see Section 9 for production hardening notes. Base URL in this report: `https://anydoc-api.onrender.com`.

3.1 GET /stats

`GET /stats`

Returns live corpus and pipeline configuration. Used by the dashboard's Overview tab and as a lightweight health check.

Request: none.

Response 200 OK:

```
{
  "document_count": 28,
  "embedding_model": "sentence-transformers/all-MiniLM-L6-v2",
  "reranker_model": "Xenova/ms-marco-MiniLM-L-6-v2",
  "reranker_enabled": true,
  "hybrid_k": 60,
  "top_n": 5
}
```

3.2 POST /ingest/url

`POST /ingest/url`

Crawls a URL (and up to 5 same-domain linked pages), chunks, and indexes it. Rejected before any network call if the resolved address is private/loopback/link-local or the AWS metadata address.

Request: application/json

```
{
  "url": "https://example.com/docs"
}
```

Response 200 OK:

```
{
  "message": "Successfully ingested 3 chunks from URL."
}
```

Error 400 (application/json): `{"detail": "URL points to an internal/private IP address: 169.254.169.254"}` for an SSRF-blocked target, or `{"detail": "Failed to read content from URL."}` if *robots.txt* disallows the crawl or the page fails to parse.

3.3 POST /ingest/file

`POST /ingest/file`

Uploads a file (PDF, DOCX, TXT, CSV; 10MB default limit) for chunking and indexing. Validated by size and by magic-byte MIME sniffing, not by filename extension.

Request: multipart/form-data, field `file`.

Response 200 OK:

```
{
  "message": "Successfully ingested 5 chunks from file."
}
```

Error 400: `{"detail": "..."}` on a failed magic-byte/size check or unparseable file.

3.4 POST /query

`POST /query`

Runs the full pipeline: query rewrite → hybrid retrieval (RRF) → cross-encoder rerank → grounded generation with citation/guardrail verification.

Request: application/json

```
{
  "query": "What ticket number was resolved by rotating the key?"
}
```

Response 200 OK:

```
{
  "answer": "SUP-90210 [1]",
  "sources": [
    {
      "id": 1,
      "source": "doc.txt",
      "content": "...",
      "rerank_score": 4.72
    }
  ],
  "search_query": "ticket number resolved by key rotation",
```

```
  "pipeline": {
    "candidates_considered": 5,
    "reranked": true,
    "elapsed_ms": 1078.1
  },
  "timings": {
    "rewrite_ms": 246.2,
    "retrieval_ms": 1078.1,
    "gen_ms": 264.0
  }
}
```

`sources[].rerank_score` is `null` when no reranker is attached. A guardrail-triggered refusal (prompt injection, profanity, or a hallucinated citation) still returns **200 OK** with a fixed refusal string in `answer` and `sources: []` - refusals are a valid pipeline outcome, not an error.

3.5 Error Response Shape

Handled errors return FastAPI's standard `{"detail": "..."}` JSON shape with a 4xx status. Clients should not assume every non-200 response is JSON, however: a reverse proxy in front of the API (e.g. on a request timeout) can return a plain-text or HTML error page instead. This distinction is not academic - it caused a real production incident, documented as Case 2 in Section 5.

4. Deployment and CI/CD

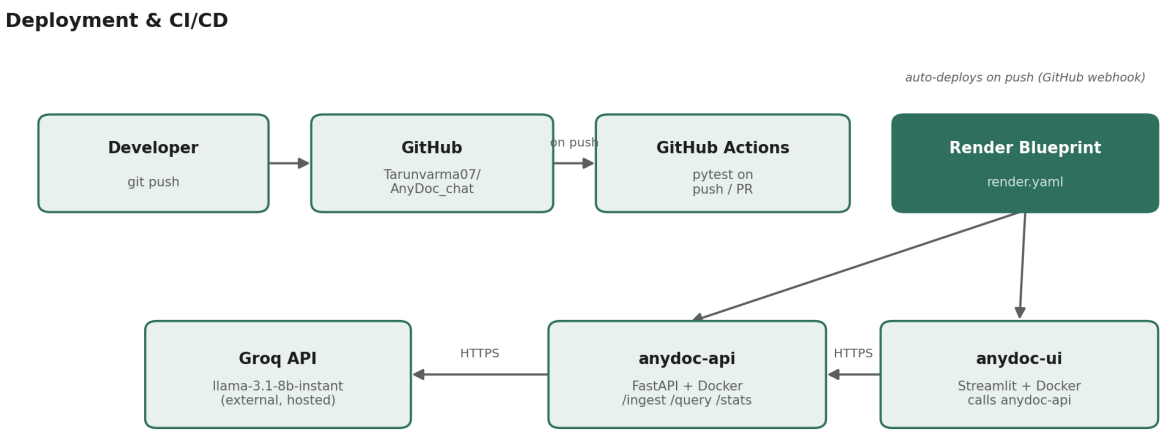


Figure 4. Push to main triggers CI (pytest) and, independently, a Render Blueprint auto-deploy of both services.

The backend (`anydoc-api`) and the Streamlit dashboard (`anydoc-ui`) are two Web Services defined in a single `render.yaml` Blueprint, built from the same Dockerfile and differentiated only by their container command. GitHub Actions runs the full `pytest` suite on every push and pull request against `main`, independently of the deploy trigger, so a push that breaks tests is visible in CI even though it does not block the Render auto-deploy. Secrets (`GROQ_API_KEY`) are entered directly in the Render dashboard, never committed; `.dockerignore` excludes `.env`, `venv/`, and `chroma_db/` from the build context.

The free-tier deployment has two known, documented constraints: (1) no persistent disk is attached, so ChromaDB's index is wiped on every redeploy or crash-triggered restart; and (2) as of this writing, the Streamlit frontend is subject to intermittent out-of-memory restarts under concurrent sessions on the 512MB free instance -

Streamlit's own dependency footprint (pandas, pyarrow, Altair) combined with per-session server-side state appears to be the driver (measured base import cost is only ~124MB, so the failure is session-accumulation under load, not a fixed baseline). The FastAPI backend is unaffected and runs reliably after the fixes in Section 5. The straightforward remediation - upgrading the affected service off the free tier - requires no code change.

5. Production Incidents: Diagnosis and Fixes

Three incidents were encountered deploying AnyDoc to a memory- and timeout-constrained hosting platform (Render's free tier). Each is documented here with its symptom, root cause, fix, and verification, because the diagnosis process - not just the fix - is the more transferable engineering artifact.

Case 1: Backend OOM at Startup

Symptom: `anydoc-api` failed to deploy with "Ran out of memory (used over 512MB)" before serving a single request.

Diagnosis: `sentence-transformers` pulls in PyTorch, whose baseline runtime footprint alone was enough to exceed the limit. A first memory measurement in the local dev environment showed an alarming ~718MB even after switching away from PyTorch - which turned out to be a false alarm: leftover `torch`/`transformers` packages, never uninstalled after the migration, were being transitively imported by `langchain-groq`/`langchain-text-splitters`, which probe for optional integrations at import time. A clean virtual environment built strictly from `requirements.txt` - matching what Render's Docker build actually installs - measured ~364MB peak, comfortably under budget.

Fix: Replaced `sentence-transformers`/PyTorch with fastembed (ONNX runtime) for both the embedding model and the cross-encoder reranker - same model weights, no PyTorch dependency.

Verification: Clean-room memory measurement (364MB peak) plus a live production test: the exact ingest-then-query sequence that previously crashed the container now returns a correct, cited answer with the document count intact afterward, confirming no restart occurred.

Case 2: Multi-Page Ingestion Timeout

Symptom: Uploading a multi-page PDF crashed the Streamlit UI with an unhandled `JSONDecodeError` instead of showing an error message.

Diagnosis: Two stacked bugs. First, `app.py` assumed every non-200 API response was FastAPI's `{"detail": ...}` JSON shape; a hosting-platform proxy returning a plain-text gateway-timeout page instead crashed the whole app on `res.json()`. Second, and the actual root cause: `PyPDFLoader` returns one `Document` per PDF page, and metadata extraction ran a separate synchronous Groq LLM call per page before chunking even started - a 12-page PDF meant 12 sequential API calls, easily enough to exceed the platform's request timeout.

Fix: Made the UI degrade to a readable error message instead of crashing on a non-JSON response. Rewrote metadata extraction to run once per uploaded file (building one combined content preview) instead of once per page/sub-document, since title/summary/topics are whole-document concepts, not per-page ones.

Verification: A unit test asserts exactly one LLM call fires for a synthetic 12-document set, with the resulting metadata applied to all 12. Live test against production: a 5-page PDF ingests in ~11 seconds (down from a per-page-multiplied duration that previously timed out).

Case 3: Guardrail False Positive from Fuzzy Matching

Symptom: A fully correct, grounded answer was refused in production with "The generated answer did not meet safety guidelines" - the output profanity guardrail firing on innocuous text.

Diagnosis: The profanity check used `better_profanity`, a maintained library chosen specifically to avoid a hand-rolled wordlist. Its matcher normalizes/strips whitespace to catch spaced-out evasion (e.g. "t e s t"), which means two entirely unrelated ADJACENT words can concatenate into a flagged term: the phrase "...for this test is

ANYDOC-..." was flagged because "test" + "is" collapsed into "testis" (a listed anatomical term). Verified directly: `contains_profanity("test is")` returns `True`; `contains_profanity("test")` and `contains_profanity("is")` individually both return `False`.
Fix: Reverted to a self-maintained profanity wordlist matched with strict `\b...\b` word-boundary regex against the literal text - no whitespace normalization, so adjacent-word concatenation is structurally impossible. This is a worse failure mode than the naive placeholder list it had originally replaced (see the methodology note below), because it silently blocks correct answers rather than simply failing to catch something.
Verification: Regression tests lock in the exact failing phrase ("this test is ANYDOC-...") alongside the original Scunthorpe-style case ("assessment" must not match on "ass"). Re-ran the exact production query end-to-end; the correct, cited answer is now returned.

Note on methodology: this project's profanity guardrail went through three iterations - a literal placeholder list, a maintained library with an unexpected failure mode, and finally a small self-maintained list with strict matching semantics. The general lesson generalizes beyond this one guardrail: "use a maintained library instead of reinventing a list" is good default advice, but it is not a substitute for understanding a dependency's actual matching semantics before trusting it in a path that can silently block correct output.

6. Evaluation

6.1 Methodology

A 105-document synthetic corpus stress-tests exact-match recall under semantic distraction: eight target facts (an order ID, a support ticket number, a config value, etc.) alongside 90 adversarial distractor documents that are topically and lexically similar to the targets but do not contain the answer. For each of eight held-out queries, we measure whether the target fact appears in the top results under (a) dense-only retrieval (top-2 of 5) and (b) hybrid retrieval (top-2, RRF-fused). Reproducible via `python src/evaluation/evaluator.py`.

6.2 Results

Retrieval Mode	Hit Rate	Notes
Dense-only (vector search)	8 / 8 (100%)	Target ranked in top 2 on every query
Hybrid (Dense + BM25, RRF-fused)	8 / 8 (100%)	No regression vs. dense-only

Table 1. Dense-only vs. hybrid retrieval hit rate on the 105-document adversarial benchmark (n=8 queries).

At this corpus size both modes saturate at 100% - itself informative: it confirms the hybrid architecture is correctly wired and introduces zero regression relative to dense-only, while remaining positioned to pay off as the corpus scales and exact-match tokens become proportionally rarer in the dense embedding space. Evaluating on a corpus one to two orders of magnitude larger is the most direct way to surface a measurable dense/hybrid gap.

7. Testing Strategy

The test suite (pytest, 25 tests across 6 files) covers chunking quality, both guardrail classes (prompt-injection and profanity, including the Case 3 regression), SSRF/file-validation guardrails, hybrid search fusion and BM25 index consistency, reranker scoring and candidate-pool sizing, query rewriting (including its fallback paths), and the metadata-extraction batching fix from Case 2. All external calls - the Groq LLM and the embedding/reranker models - are mocked at the unit level; the reranker and embedding tests that need a real model run against a temporary on-disk Chroma collection rather than the live index. GitHub Actions runs the full suite on every push and pull request against `main`.

8. Related Work

AnyDoc builds directly on retrieval-augmented generation as introduced by Lewis et al. (2020). The sparse side of the hybrid retriever follows the BM25 probabilistic relevance framework (Robertson and Zaragoza, 2009), and the fusion method follows Reciprocal Rank Fusion (Cormack et al., 2009), which outperforms individual rank-learning methods without requiring score calibration between retrievers - a practical advantage when combining a cosine-similarity dense retriever with a BM25 sparse retriever whose scores are not directly comparable. The reranking stage follows Nogueira and Cho (2019). The dense embedding model is a Sentence-Transformers model (Reimers and Gurevych, 2019), run on ONNX rather than PyTorch specifically for the deployment-memory reasons documented in Case 1. We are not aware of prior RAG system write-ups that discuss SSRF as an ingestion-time attack surface alongside a full account of production deployment failures; most security-focused RAG literature concentrates on prompt injection and data poisoning, which AnyDoc also mitigates at both the input and output stage.

9. Limitations and Future Work

- The evaluation corpus (105 documents, 8 queries) is small; both retrieval modes saturate at 100%, so it does not yet demonstrate a measurable hybrid advantage. A larger benchmark (e.g. a Natural Questions or HotpotQA subset) would better separate the two approaches.
- No authentication/authorization on the API - every endpoint is public in the current deployment. A production hardening pass would add an API key or OAuth layer, rate limiting per client, and per-tenant document isolation (the index is currently a single shared collection).
- No persistent disk on the current free-tier deployment; the vector index does not survive a redeploy or crash-triggered restart.
- Retrieved chunk metadata (document type, extracted topics) is captured at ingestion time but not yet used to filter or boost retrieval at query time.
- Generation is synchronous and non-streamed; streaming (Server-Sent Events or WebSocket) would improve perceived latency for longer answers.
- Guardrails use pattern-based prompt-injection detection and a maintained-but-hand-curated profanity wordlist rather than a dedicated moderation model; a production deployment would likely integrate a hosted moderation API.
- The Streamlit frontend's memory behavior under concurrent sessions on constrained hosting (Section 4) is not yet fully root-caused to the same rigor as Cases 1-3; the working hypothesis (per-session state accumulation) has not been isolated with a clean-room measurement the way Case 1's PyTorch footprint was.

10. Conclusion

AnyDoc demonstrates a hybrid dense/sparse retriever with RRF fusion, cross-encoder reranking, and LLM-based query rewriting, assembled into a documented, versioned API and shipped through a real CI/CD pipeline to a hosting platform - not just a local notebook. The three production incidents in Section 5 were each root-caused with a concrete measurement (a clean-room memory profile, a call-count assertion, a direct library-behavior test) rather than guessed at, and each fix is locked in by a regression test. The natural next steps are evaluating retrieval at a scale where dense-only degrades measurably, and closing the authentication and multi-tenancy gaps noted in Section 9 before any real production use.

References

Cormack, G. V., Clarke, C. L. A., and Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, pp. 758-759.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)* 33.

Nogueira, R. and Cho, K. (2019). Passage Re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.

Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982-3992.

Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), pp. 333-389.